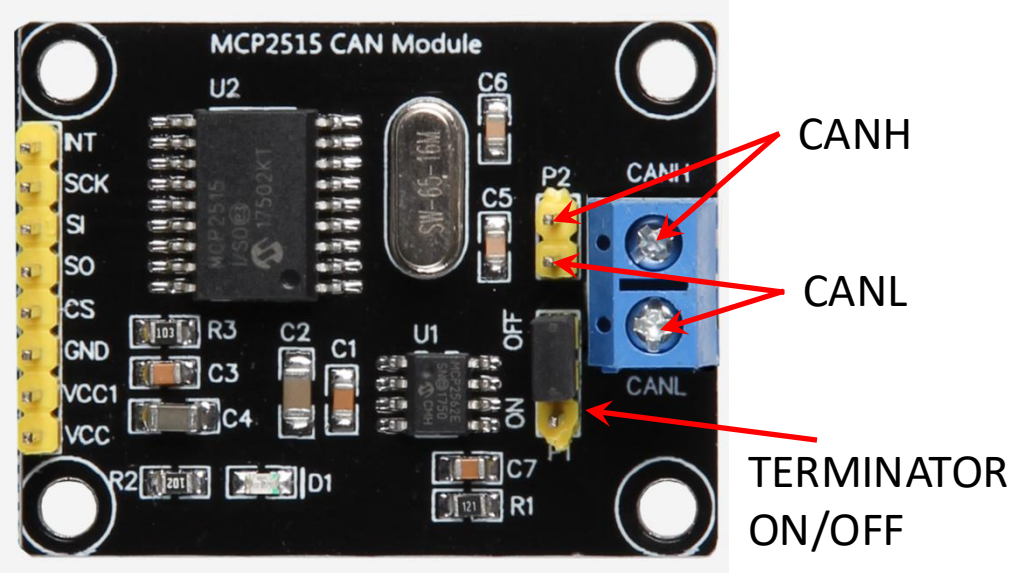
Distributed Real-Time Control Systems

Module 10

The CAN-BUS Programming

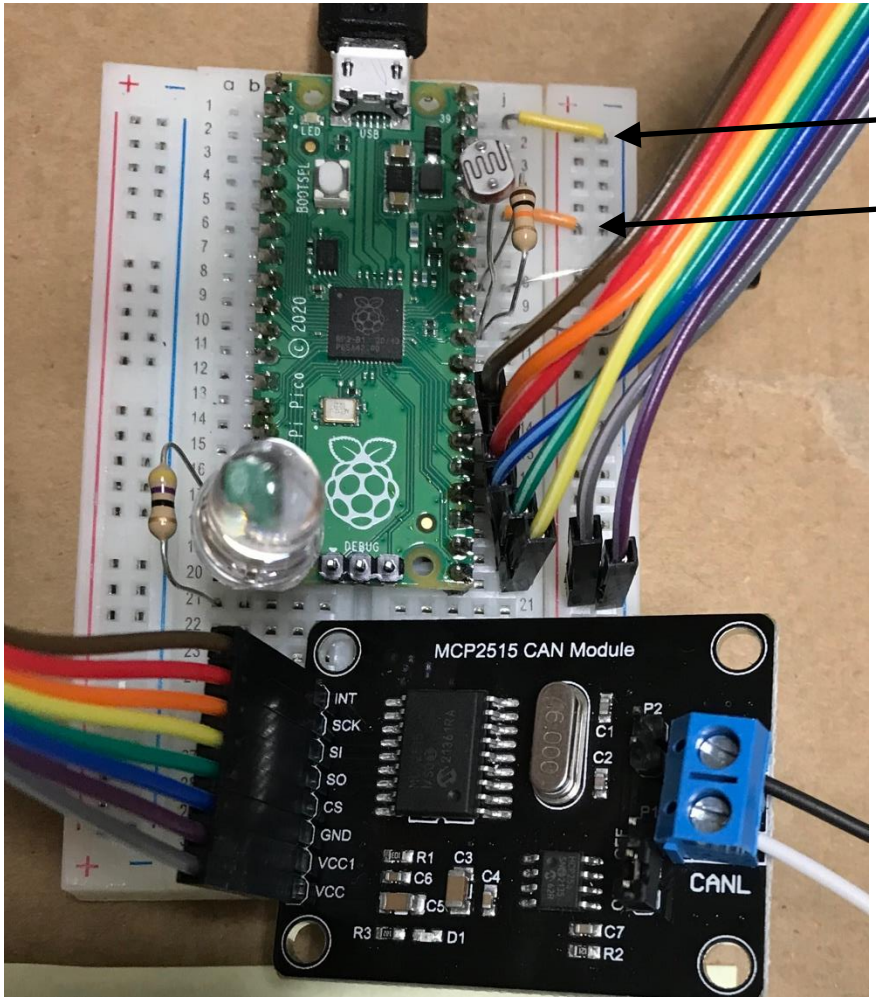
The MCP2515 CAN Module



CAN-MODULE	PICO
VCC	3V3OUT (3.3V)
VCC1	VBUS (5V)
GND	GND
CS	SPIn CSn
SO	SPIn RX
SI	SPIn TX
SCK	SPIn SCK
INT	GPIOn

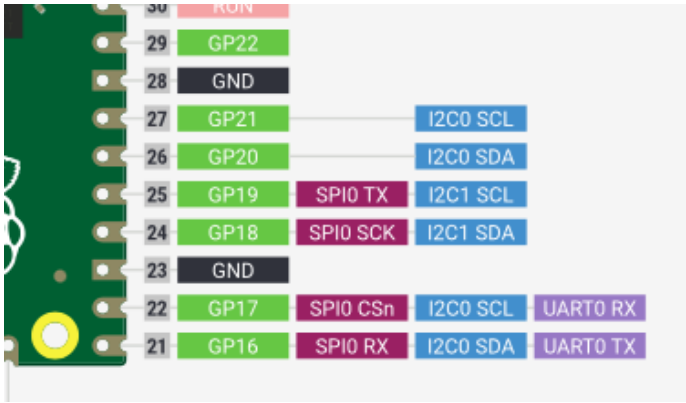
- Implements CAN V2.0B at 1MB/s
- **High Speed SPI Interface (10Mz), modes 0,0 and 1,1**.
- Interrupt output pin
- Low power CMOS (2.7V-5.5V), 5mA current (typ).
- Put terminator ON if node is in an extremity of the bus.

Possible Connection Scheme



VBUS = 5V

3V3OUT = 3.3V



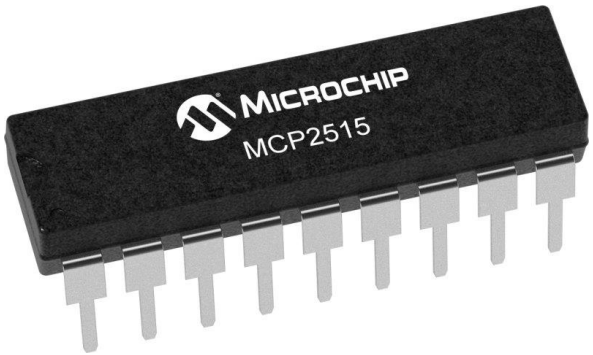
The MCP2515

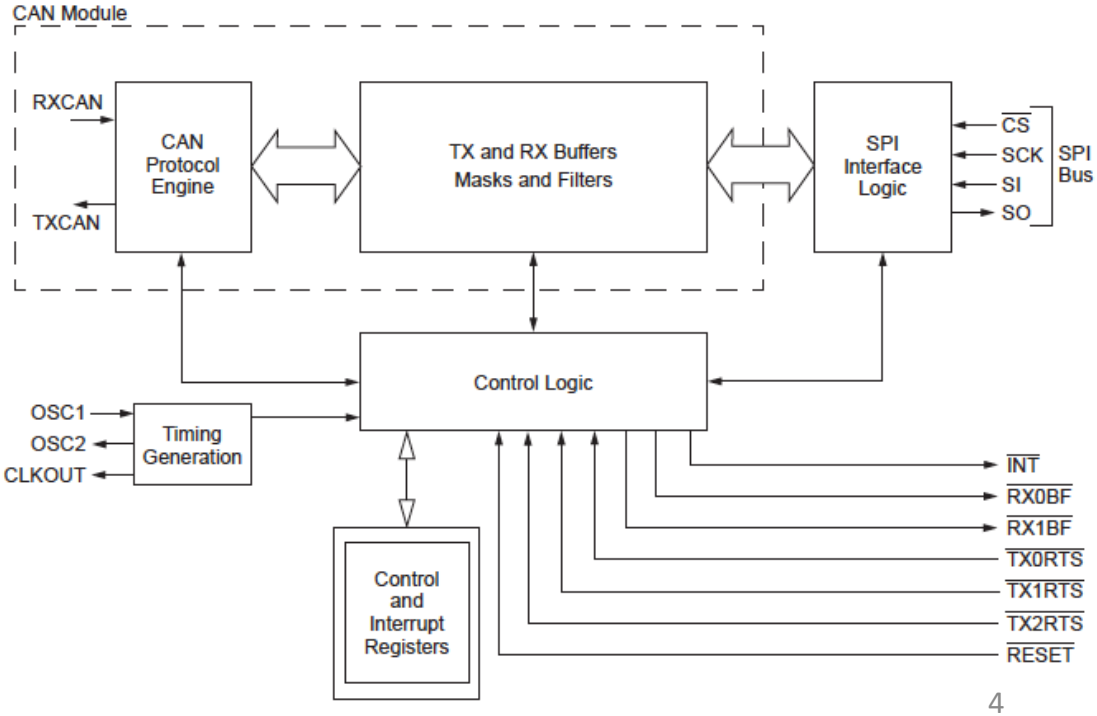
Device Overview

The MCP2515 CAN module handles all functions for receiving and transmitting messages on the CAN bus.

Transmission and reception of data is managed by TX and RX buffers with message filtering.

The configuration and operation of the device is done by reading/writing device registers via spi.





MCP2515 Programming

- Download the Rasperry Pi Pico MCP2515 CAN Interface Library:
  - https://github.com/adamczykpiotr/pico-mcp2515
- Use files “can.h”, ”mcp2515.h” and ”mcp2515.cpp” in your project folder or as a library in the Arduino IDE.

MCP2515 Pico Class

Constructor:

```
MCP2515 can0 {channel, CSn, TX, RX, SCK, CLK_SPEED}; //channel spi0 or spi1
```

Initialization Example:

```
can0.reset();
can0.setBitrate(CAN_1000KBPS);
can0.setNormalMode();
//or setLoopbackMode() for debug
```

Sending Messages:

```
struct can_frame msg;
//fill msg fields
can0.sendMessage(&msg);
```

Can Message Structure:

```
struct can_frame {
  uint32_t can_id; //msg id
  uint8_t can_dlc; //msg size
  uint8_t data[8]; //msg data
};
```

Receiving Messages:

```
struct can_frame msg;
can0.readMessage(&msg);
//use received data
```

Error codes:

```
MCP2515::ERROR { ERROR_OK, ERROR_FAIL, ERROR_ALLTXBUSY, ERROR_FAILINIT, ERROR_FAILTX, ERROR_NOMSG};
```

Pico MCP2515 Class

Initialization Functions

```
ERROR reset( void );
ERROR setConfigMode();                  //Initialization
ERROR setListenOnlyMode();              //Only reads the bus, does not write
ERROR setSleepMode();                   //To minimize energy consumption
ERROR setLoopbackMode();                //For testing - simulate bus operation
ERROR setNormalMode();                  //Read and Write normally
ERROR setBitrate( const CAN_SPEED canSpeed );
```

Return: MCP2515::ERROR_OK or
MCP2515::ERROR_FAIL

Choosing Frame Types in MCP2515 Pico Lib

```
struct can_frame {
   uint32_t can_id;   // 11 or 29 bit ID + CAN_EFF_FLAG + CAN_RTR_FLAG
   uint8_t  can_dlc;  // number of data bytes (max 8)
   uint8_t  data[8];  // buffer holding the data bytes
};
```

can_id

Bit 31	Bit 30	Bit 29	Bits 11-28	Bits 0-10
EFF	RTR	0	Extended ID	ID

```
/* special address description flags for the CAN_ID */
#define CAN_EFF_FLAG 0x80000000UL /* EFF/SFF is set in the MSB */
#define CAN_RTR_FLAG 0x40000000UL /* remote transmission request *//

/* valid bits in CAN ID for frame formats */
#define CAN_SFF_MASK 0x000007FFUL /* standard frame format (SFF) */
#define CAN_EFF_MASK 0x1FFFFFFFUL /* extended frame format (EFF) */
```

Example: Periodic Read/Writes

```
#include "mcp2515.h"

pico_unique_board_id_t pico_board_id; // full id
uint8_t node_address; // short id
struct can_frame canMsgTx, canMsgRx;
unsigned long counter{0};
MCP2515::ERROR err;
uint8_t unsigned long time_to_write;
unsigned long write_delay {1000};
const int BUFSZ = 100;
char printbuf[BUFSZ];

MCP2515 can0 {spi0, 17, 19, 16, 18, 10000000};


void setup() {
  pico_get_unique_board_id(&pico_board_id);
  node_address = pico_board_id.id[7]; //check
  Serial.begin();
  can0.reset();
  can0.setBitrate(CAN_1000KBPS);
  can0.setNormalMode(); //setLoopbackMode()debug
  unsigned long current_time = millis();
  time_to_write = current_time + write_delay;
}
```

```
void loop() {
  unsigned long current_time = millis();
  if( current_time >= time_to_write ) {
    canMsgTx.can_id = node_address;
    canMsgTx.can_dlc = 8;
    unsigned long div = counter;
    for( int i = 0; i < canMsgTx.can_dlc; i++ ) {
    canMsgTx.data[i] = '0'+(int)(div%10);
    div = div/10;
  }
  err = can0.sendMessage(&canMsgTx);
  snprintf(printbuf, BUFSZ,
    "Sending message %ld from node %x\n",
    counter++, node_address);
  Serial.print(printbuf);
  time_to_write = current_time+write_delay;
  }
  while ( (err = can0.readMessage( &canMsgRx )) ==
     MCP2515::ERROR_OK ) {
    unsigned long rx_msg = 0;
    unsigned long mult = 1;
    for (int i=0 ; i < canMsgRx.can_dlc ; i++) {
      rx_msg += mult*(canMsgRx.data[i]-'0');
      mult = mult*10;
    }
    snprintf(printbuf, BUFSZ,
    "\t\t\t\tReceived message %ld from node %x\n",
    rx_msg, (int)canMsgRx.can_id);
    Serial.print(printbuf);
  }
 if( err == MCP2515::ERROR_FAIL)
  Serial.println("Error");
  }
```

Example: Periodic Read/Writes

The code starts by reading the unique flash number of the pico, which will be used to create different id's of messages sent by different picos

The MCP2515 can0 object is created as a global object to be accessible from any function.

Data is converted from integer to text and vice-versa.

All functions return a MCP2515::ERROR code that should be tested for errors.

The MCP2515

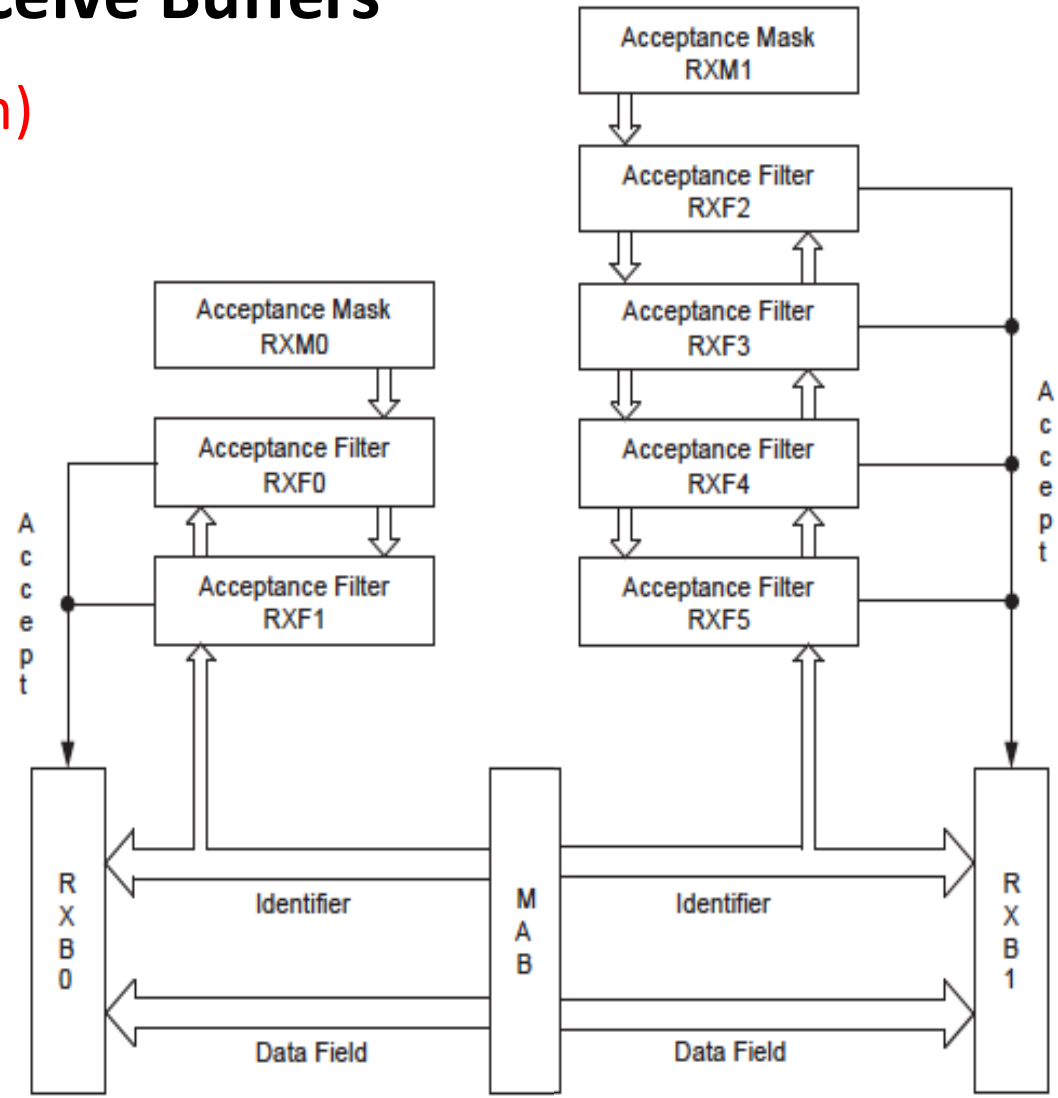
Receive Buffers

- Two Receive Buffers (RXBn)
- One acceptance mask for each buffer.
- 2 filters for RXB0 and 4 filters for RXB1
- RXB0 has higher priority

TABLE 4-2: FILTER/MASK TRUTH TABLE

Mask Bit n	Filter Bit n	Message Identifier Bit	Accept or Reject Bit n
0	x	x	Accept
1	0	0	Accept
1	0	1	Reject
1	1	0	Reject
1	1	1	Accept

Note: x = don't care



Pico MCP2515 Class

Transmit and Receive Functions

```
// Types defined in class MCP2515 (mcp2515.h)
// enum RXBn   { RXB0, RXB1 };                    //receive buffers
// enum TXBn   { TXB0 = 0, TXB1 = 1, TXB2 = 2 };  //transmit buffers
```

```
ERROR sendMessage( const TXBn txbn, const struct can_frame *frame );
ERROR sendMessage( const struct can_frame *frame );
```

Return:
MCP2515::ERROR_OK or
MCP2515::ERROR_FAILTX or
MCP2515::ERROR_ALLTXBUSY

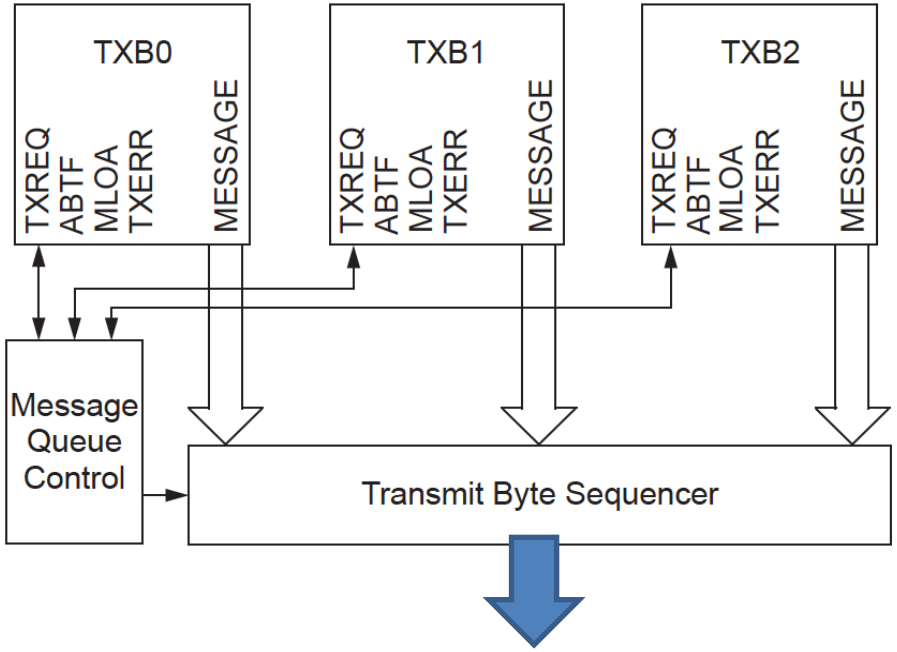
```
ERROR readMessage( const RXBn rxbn, struct can_frame *frame);
ERROR readMessage( struct can_frame *frame );
bool checkReceive( );
```

Return:
MCP2515::ERROR_OK or
MCP2515::ERROR_NOMSG or
MCP2515::ERROR_FAIL

The MCP2515

Transmit Buffers

- Three Transmit Buffers (TXBn)
- Each buffer has 14 bytes:
  - First byte TXBnCTRL (Control flags, priority)
  - Five bytes for ID (standard or extended), Data length.
  - 8 bytes data
- Transmit Priority
  - Highest priority first
  - If equal priority, lowest n first (e.g. TXB0 before TXB1)



MCP2515 Library

Masks and Filters

- Definition of Masks and Filters must be done in configuration mode - before the call to any of the SetModeXXX() functions.

```
ERROR setFilterMask( const MASK num, const bool ext, const uint32_t ulData);
ERROR setFilter( const RXF num, const bool ext, const uint32_t ulData);
```

Return: MCP2515::ERROR_OK or
MCP2515::ERROR_FAIL

- Mask bits set to '1' indicate that the corresponding bits on the filters should be checked for a match.
- If any of the bits of a message ID in the masked positions matches the filter bits in the same positions, the message is accepted.
- There can be multiple matches. The first successful one accepts the message.

MCP2515 Library

Masks and Filters

- Example: If a node wants to accept individual messages should define the mask as 0xFFFFFFFF, and the filters to the ID of the desired messages. Then the filter value can be interpreted as the “address” of the node.

- Example: If a node wants to accept all messages whose IDs are even, should define the mask as 0x00000001, and the filter as 0x*******0. (* - don’t care)

Example: setting Masks and Filters

```
// Types defined in class MCP2515 (mcp2515.h)
// enum MASK { MASK0, MASK1 };                          //reception masks
// enum RXF    { RXF0, RXF1, RXF2, RXF3, RXF4, RXF5 };   //reception filters

void setup() {
  mcp2515.reset();
  mcp2515.setBitrate(CAN_1000KBPS, MCP_16MHZ);
  mcp2515.setFilterMask(MCP2515::MASK0, 0, 0xFFFFFFFF);   //RXBUFFER0
  mcp2515.setFilter(MCP2515::RXF0, 0, 0x0000000A);        //accepts msg ID 0x0000000A
  mcp2515.setFilter(MCP2515::RXF1, 0, 3);                 //accepts msg ID 0x00000003
  mcp2515.setFilterMask(MCP2515::MASK1, 0, 0x0000000F);   //RXBUFFER1
  mcp2515.setFilter(MCP2515::RXF2, 0, 0x0000000A);        //accepts msg with ID ending with 1010
  mcp2515.setFilter(MCP2515::RXF3, 0, 3);                 //accepts msg with ID ending with 0011
  mcp2515.setFilter(MCP2515::RXF4, 0, 0);                 //accepts msg with ID ending with 0000
  mcp2515.setFilter(MCP2515::RXF5, 1, 0);                 /accepts extended frames with ID ending with 0000
  mcp2515.setNormalMode();
}
```